



```
$ python manage.py makemigrations polls
```



```
...> py manage.py makemigrations polls
```

After you run the above command, you will get details as displayed in the image below, on your screen.



```
Migrations for 'polls':  
polls/migrations/0001_initial.py  
  - Create model Question  
  - Create model Choice
```

If you need to perform 'Migrations', then there is a command for it. With the help of this command, you can automatically manage app database schema, which is called 'migrate'. Now, we will perform an SQL migration command, i.e., 'sqlmigrate'; this command will return migration name/s and their SQL details as shown in the screenshot below (run as per your operating system):



```
...> py manage.py sqlmigrate polls 0001
```



```
$ python manage.py sqlmigrate polls 0001
```

The result will be similar to the image shown on the next page.

Hence, you can see that the 'migrate command' takes all those migrations that are not applied. How do we know whether the migration is applied or not? Well, it can be tracked by Django. There is a special table in DB known as 'django_migrations'. It has a record of all the migrations on which migration are applied. As a result, we have synchronisation of all the changes that took place on models along with the schema in the DB.

With this, it is clear that 'Migrations' are a very powerful command as it empowers a developer to alter models as and as required for any